я не приду , у меня
форс-мажор
я перехотел

Глава 9. SPI. Внутреннее устройство контроллеров передачи данных

Ежелев Г. И.

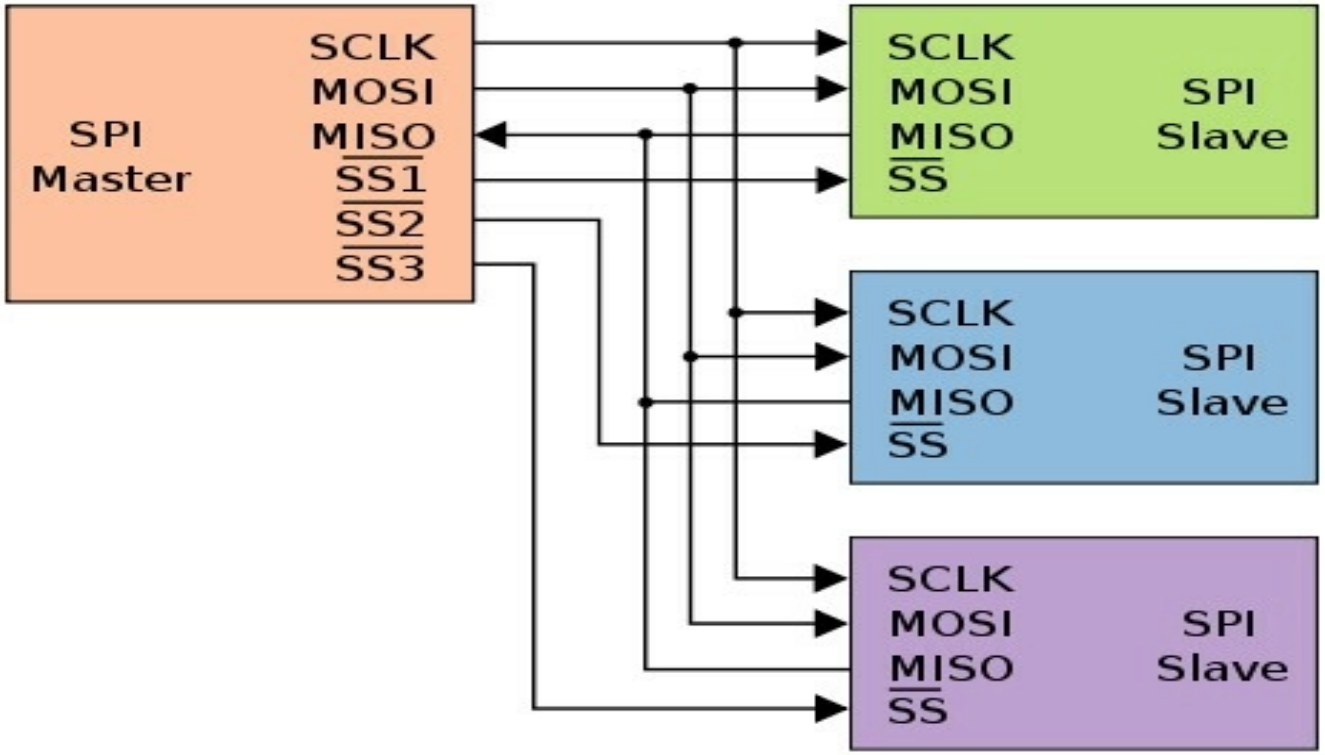
31 марта 2026 г.

Протокол SPI. Особенности.

- Концепция Master-Slave, как в IIC.
- Шина передачи данных разделена на две:
  MOSI - **Master Output Slave Input**.
  MISO - **Master Input Slave Output**.
- Такое разделение позволяет передавать данные одновременно в обе стороны.
- Такая передача данных называется полнодуплексной.
- Рассчитан на передачу на малые расстояния, поэтому в SPI, в отличие от других протоколов, почти не используются различные служебные биты, в силу низкой вероятности возникновения ошибки.

Протокол SPI. Шины.

- Шины MOSI и MISO работают похоже на то, как работает шина SDA в IIC.
- SCLK (Serial Clock) - Сигнал синхронизации в SPI.
- SS/CS (Slave Select/Chip Select) - Выбор ведомого. Простой цифровой сигнал для обозначения текущего активного ведомого устройства.



Синхронность. Режимы тактирования.

SPI - синхронный протокол - т.е. все данные передаются с помощью сигнала синхронизации(SCLK).

У SPI есть 4 основных режима работы сигнала синхронизации.

CPOL - Clock Polarity - полярность сигнала синхронизации.

- CPOL $= 0 \sim$ покой - SCLK $= 0$.
- CPOL $= 1 \sim$ покой - SCLK $= 1$.

CPHA - Clock Phase - Определяет, по какому фронту SCLK выставляются данные, а по какому читаются. Рассмотрим один такт вида: 0->1->0, где в данном случае 1 - активный уровень на SCLK, он не обязан совпадать с физическим уровнем, т.к. есть CPOL!

- CPHA = 0. Чтение на первом фронте, изменение данных на втором
- CPHA = 1. Наоборот, выставление на первом, чтение на втором.
- Понятно, что в случае CPHA = 0 первый бит должен быть выставлен заранее.

CPOL и CPHA должны совпадать у всех устройств.

Режим	CPOL	CPHA	SCLK в покое	Когда считываем данные
0	0	0	0	На фронте 0→1
1	0	1	0	На фронте 1→0
2	1	0	1	На фронте 1→0
3	1	1	1	На фронте 0→1

Длина одного пакета

- Данные передаются по сдвиговым регистрам, так же как и в IIC и UART.
- В SPI нет стартовых и стоповых последовательностей, поэтому длина одной передачи может варьироваться. Обычно число бит кратно 8.
- Протокол рассчитан на высокие частоты (1-10+ МГц), но низкое расстояние передачи.

Выводы.

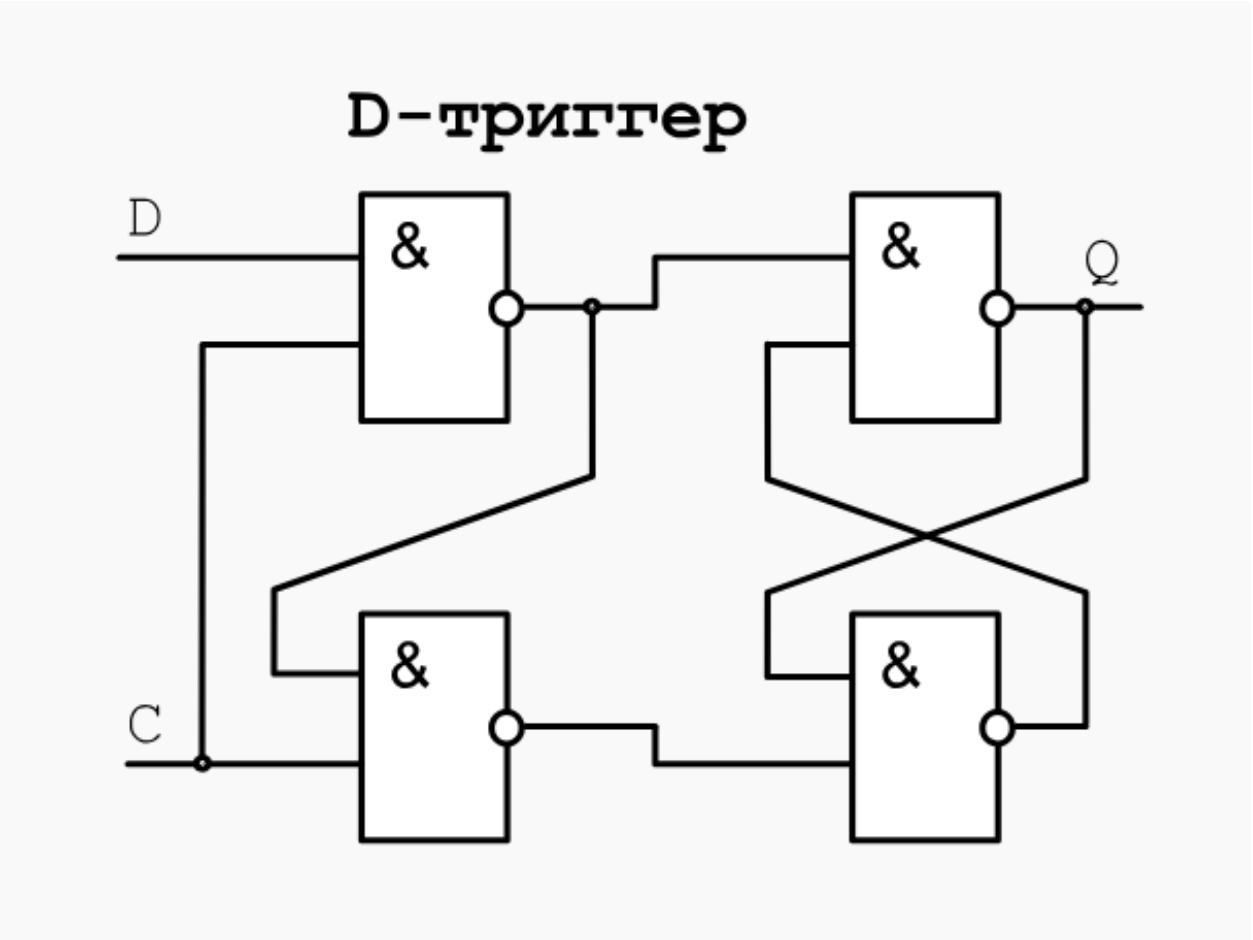
- Высокая скорость
- Простота
- Низкое расстояние передачи из-за отсутсвия дифференциальных сигналов.
- Если для вас этот минус критичен, следующий шаг - протокол USB.
- USB так же поддерживается STM32, в том числе f407.

Устройство контроллера SPI изнутри.

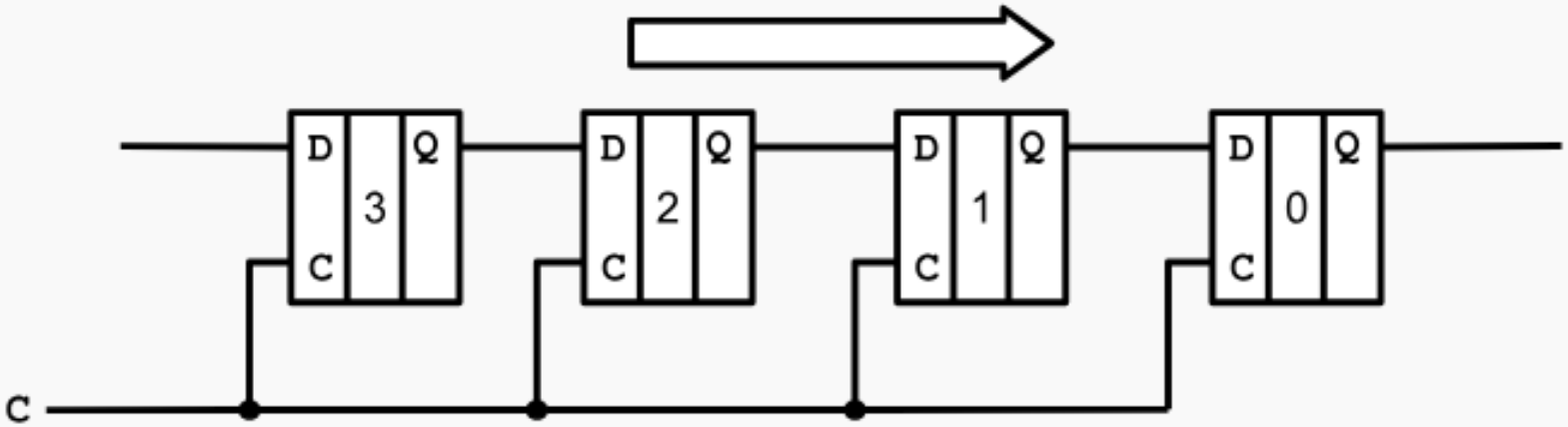
Мы никогда не смотрели внутрь контроллеров передачи данных. Это упущение, которое сейчас будет исправлено. Советую вспомнить устройство D-триггера и счётчика.

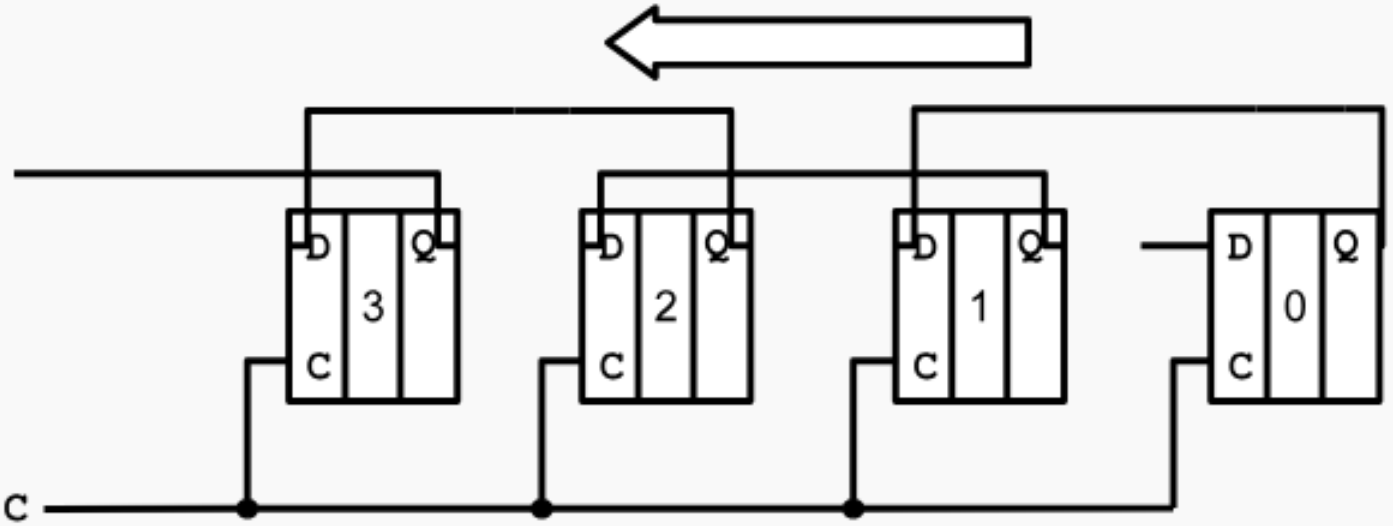
NB. ADC, контроллер SPI и тд - a.k.a - контроллеры внешних устройств

Еще один вариант триггера D-триггер.

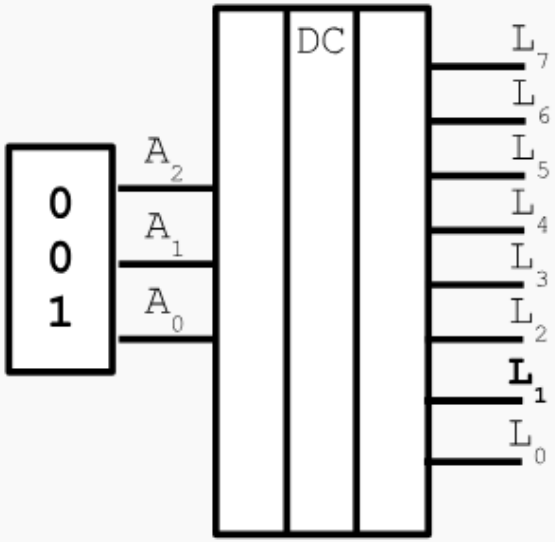


Отступление: сдвиговый регистр



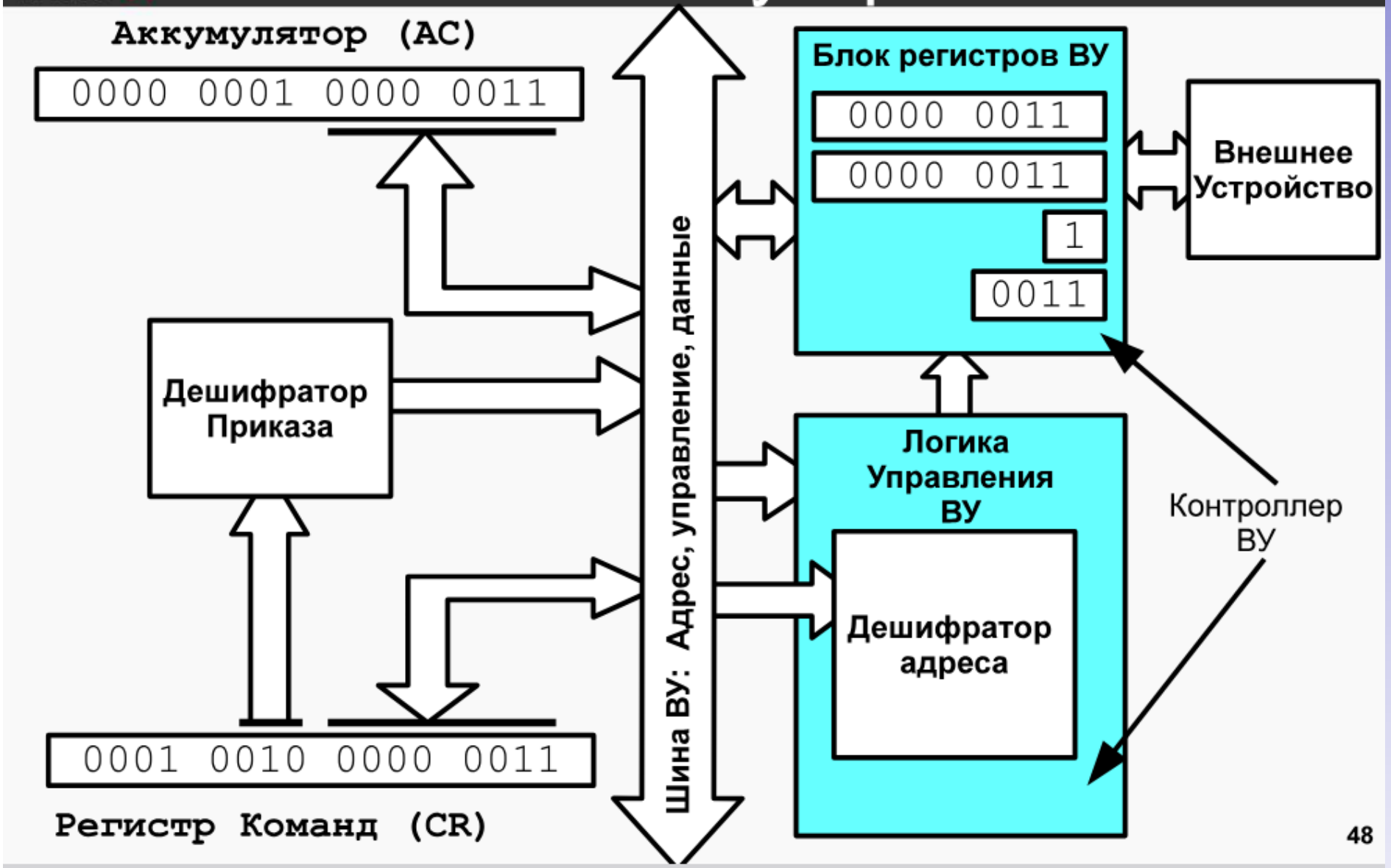


Отступление: Дешифратор



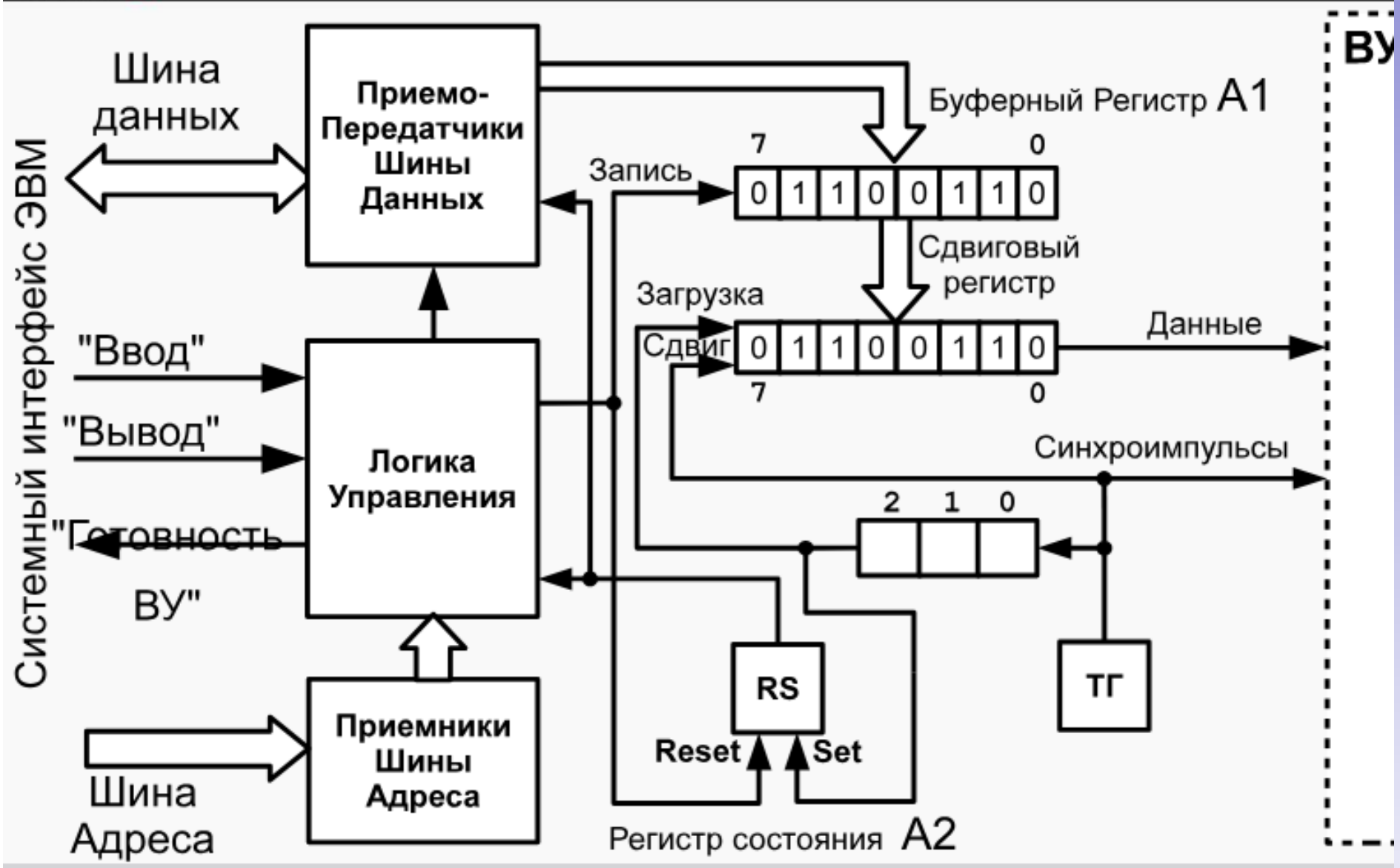
Адрес				Строка							
A_2	A_1	A_0		L_7	L_6	L_5	L_4	L_3	L_2	L_1	L_0
0	0	0		0	0	0	0	0	0	0	1
0	**0**	**1**		**0**	**0**	**0**	**0**	**0**	**0**	**1**	**0**
0	1	0		0	0	0	0	0	1	0	0
0	1	1		0	0	0	0	1	0	0	0
1	0	0		0	0	0	1	0	0	0	0
1	0	1		0	0	1	0	0	0	0	0
1	1	0		0	1	0	0	0	0	0	0
1	1	1		1	0	0	0	0	0	0	0

Шина БЭВМ-NG и внешние устройства

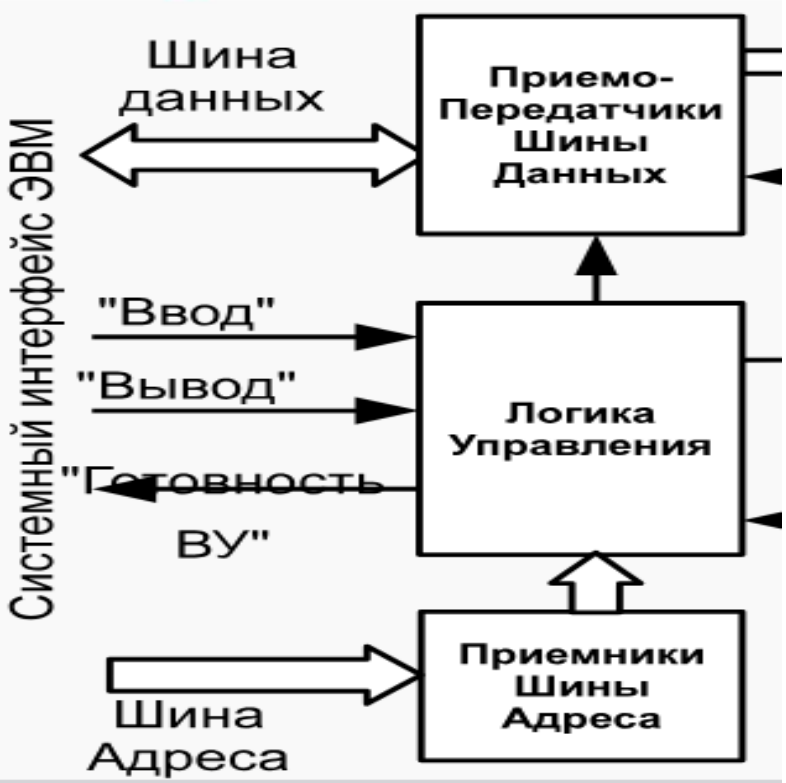


Теперь рассмотрим обобщенное устройство любого контроллера синхронного протокола передачи данных.

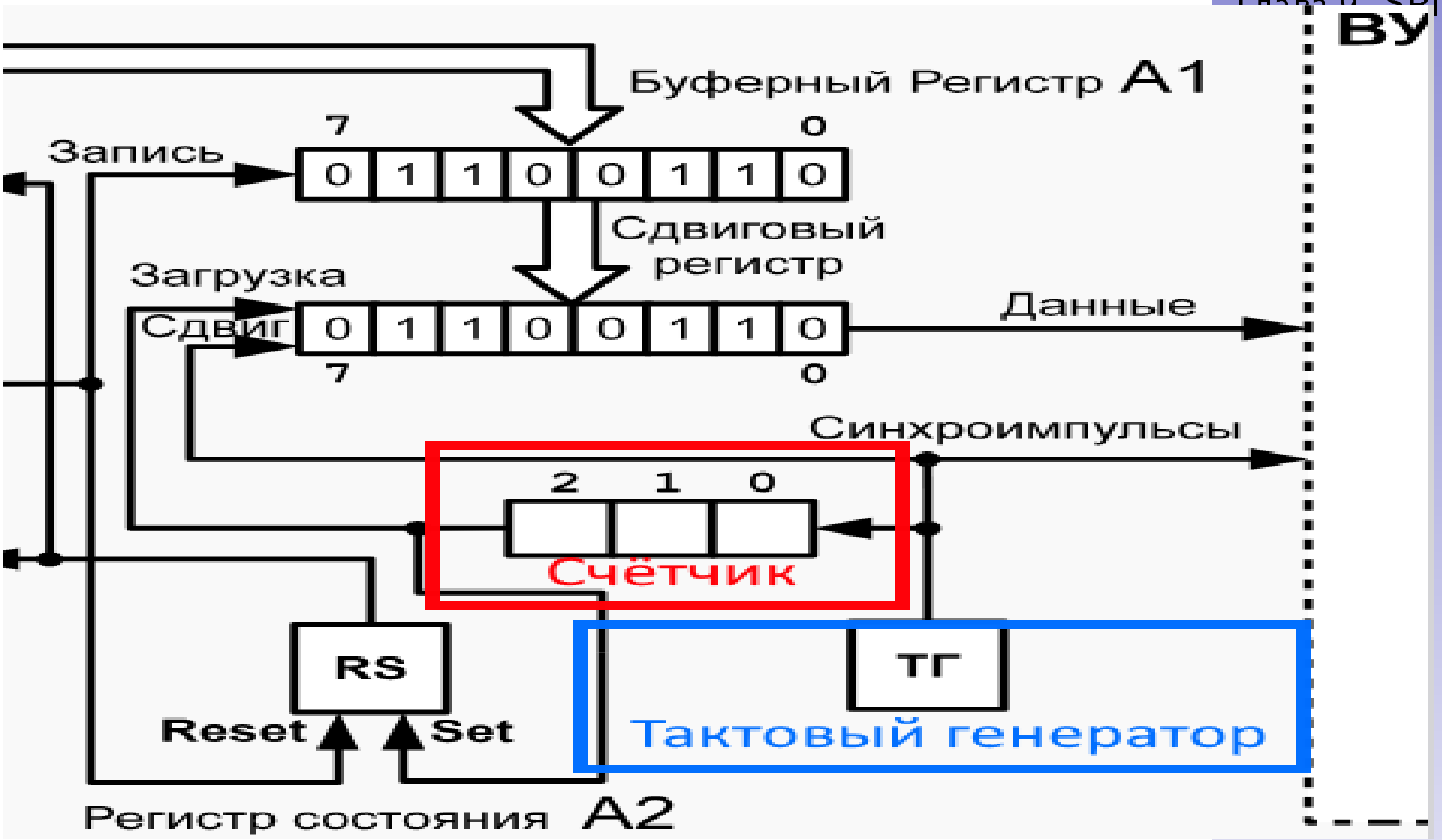
Контроллер передачи синхронного последовательного интерфейса



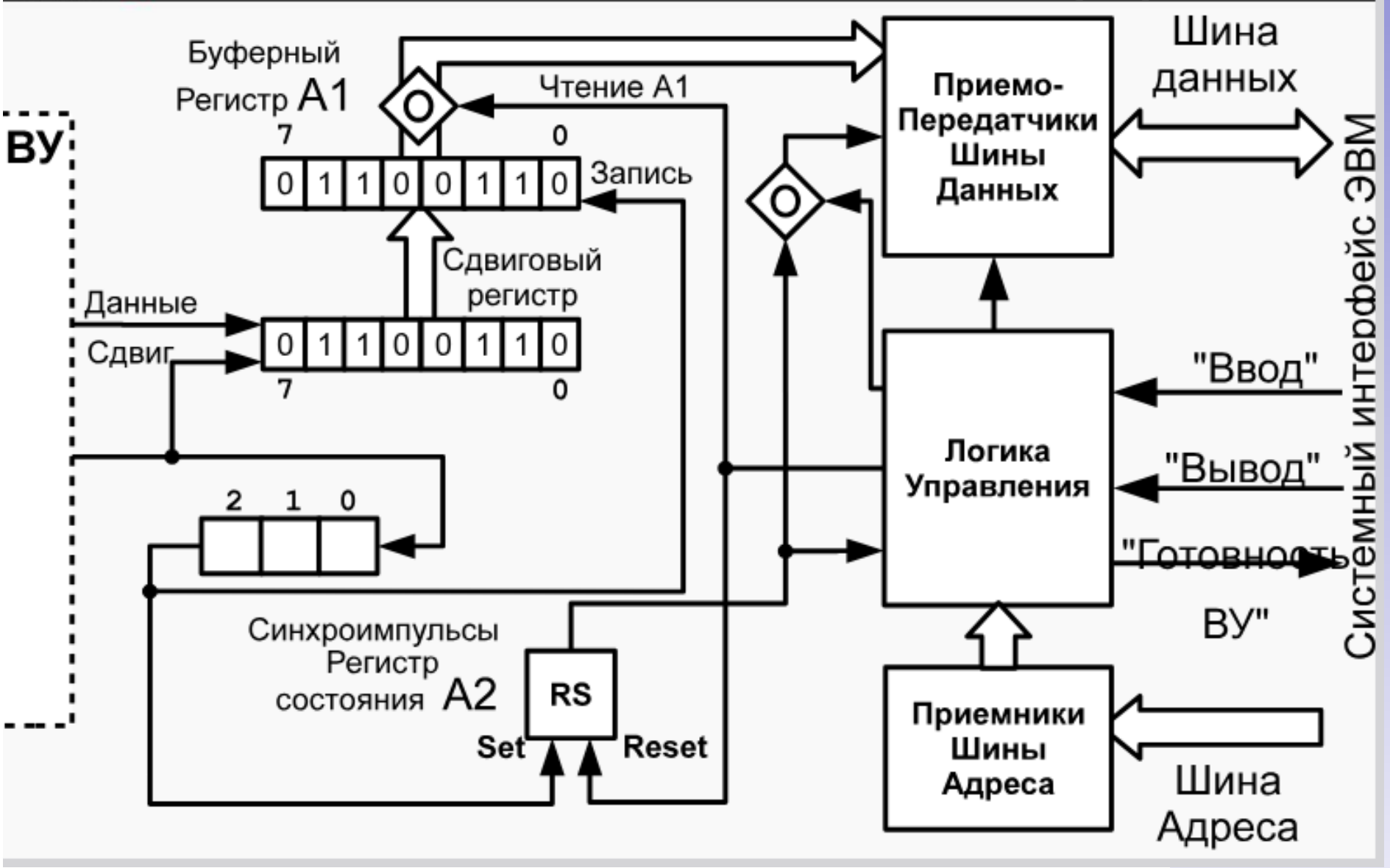
Шина данных



Обычно системные шины общего назначение, что в МК, что в ПК, разделены на шину адреса, где выставляется в двоичном виде адрес устройства ЭВМ, шину данных, где собственно данные передаются и шину управления, где передаются служебные сигналы.

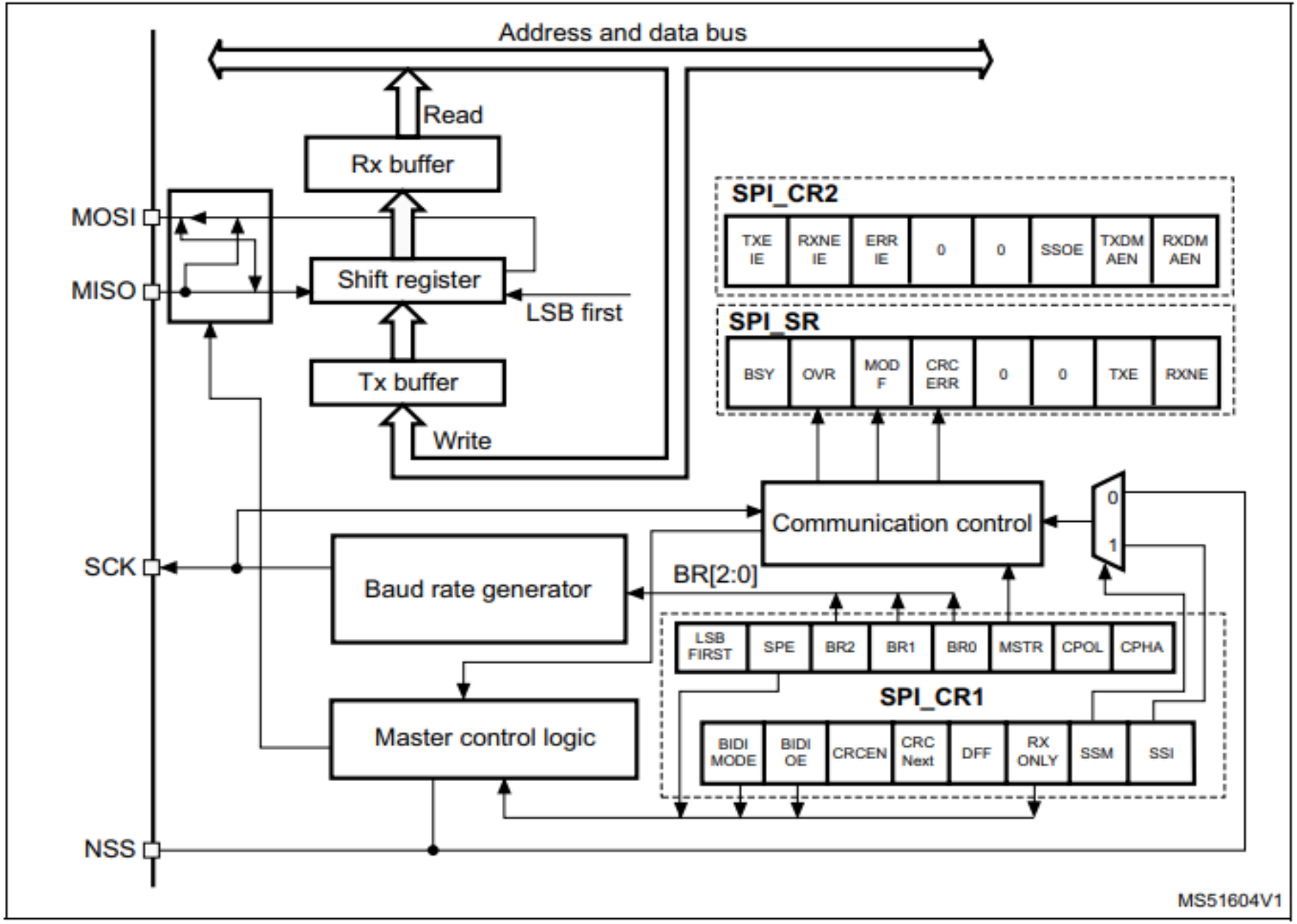

ВУ
Буферный Регистр A1
7
0
Запись
0 1 1 0 0 1 1 0
Сдвиговый регистр
Загрузка
Сдвиг
0 1 1 0 0 1 1 0
7
0
Данные
Синхроимпульсы
2 1 0
Счётчик
RS
Reset
Set
ТГ
Тактовый генератор
Регистр состояния A2

Контр. приема синхронного последовательного интерфейса

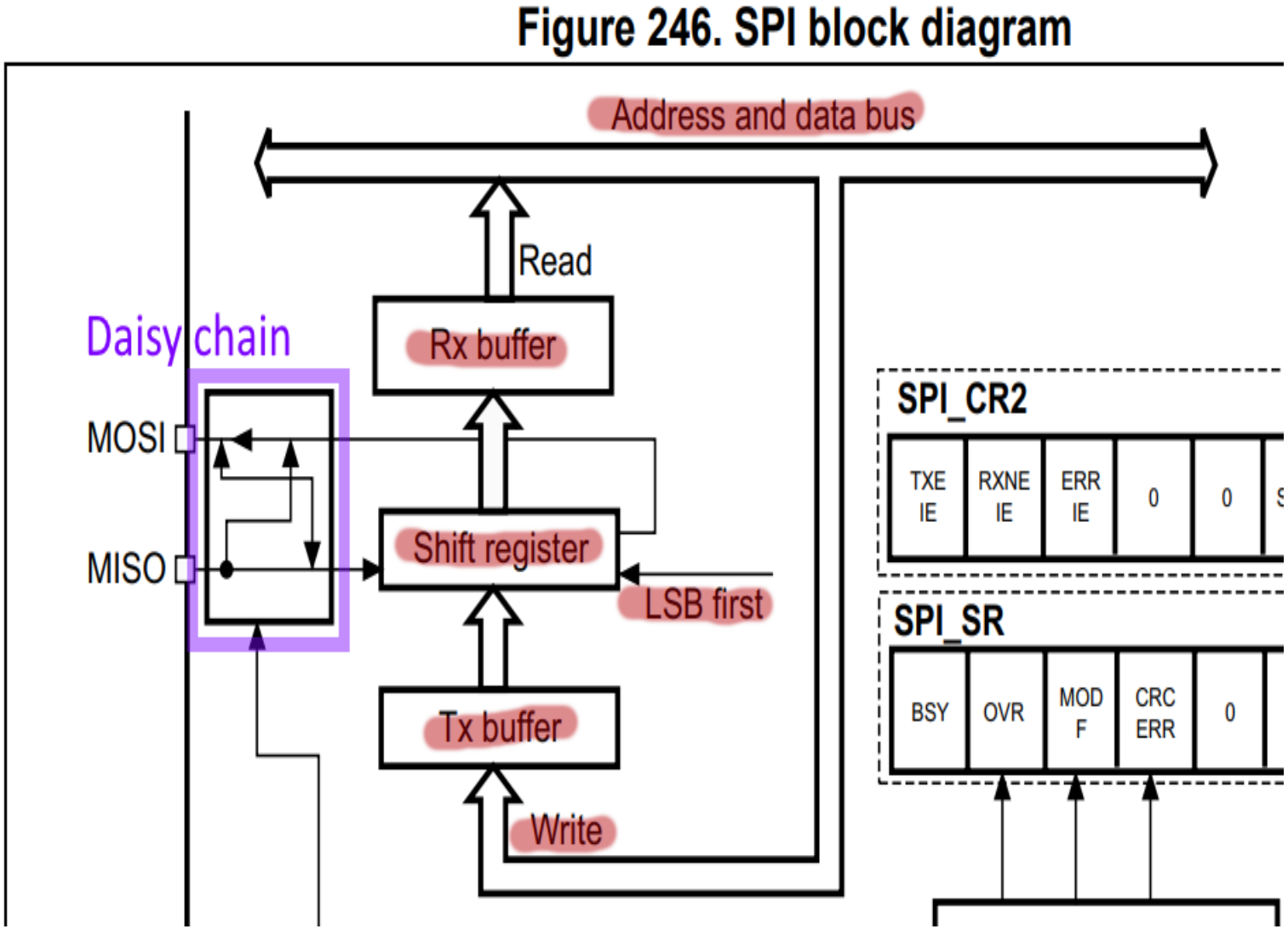


Контроллер SPI в STM32

Figure 246. SPI block diagram



Что вокруг сдвигового регистра?



Пунктирные прямоугольники - регистры SPI

- Daisy chain - Физическое замыкание входа и выхода для последовательной передачи данных между множеством подчиненных.

Код. Цепочки инициализации

До этого мы не сильно думали про последовательность инициализации (точнее за вас уже подумали).
Теперь стоит закрыть и этот пробел.

•Цепочка инициализации - правильная последовательность настроек различных устройств, чтобы не произошло конфликтов.

• Можно выделить *а.к.а глобальные* и *локальные*. цепочки. Т.е. правильные последовательности настройки различных устройств и правильные последовательности настройки конкретного устройства

Глобальные цепочки инициализации

Сначала изучим цепочку инициализации из startup файла, напомним что именно в нем лежит таблица векторов прерываний (адресов их обработчиков)

- 3 секции данных - heap, stack, vector table
- Далее идет секция кода
- Сначала вызывается функция SystemInit() - обнуление регистров, базовая настройка частот
- Вызов функции __main которая в конечном счёте станет точкой входа программы.
- Так же в этом файле лежат обработчики исключений - по факту просто бесконечные циклы, которые вызываются в случае ошибки. По тому, в какой именно цикл зашла программа можно понять тип ошибки.

- Ещё в этом файле указаны директивы для компилятора - использовать сокращенный набор команд (совместимость с ARM ядром) и 8-битный размер команд (тоже совместимость с ядром).
- Также объявлены указатели на вершину стека вызовов
- секции стека и кучи объявлены READ ONLY для защиты от случайной записи.

Локальные цепочки инициализации

Разберем на примере SPI.

Когда дело доходит до кода - Refference manual (есть в репозитории).

28.3.3 Configuring the SPI in master mode

In the master configuration, the serial clock is generated on the SCK pin.

Procedure

1. Select the BR[2:0] bits to define the serial clock baud rate (see SPI_CR1 register).
2. Select the CPOL and CPHA bits to define one of the four relationships between the data transfer and the serial clock (see *Figure 248*). This step is not required when the TI mode is selected.
3. Set the DFF bit to define 8- or 16-bit data frame format
4. Configure the LSBFIRST bit in the SPI_CR1 register to define the frame format. This step is not required when the TI mode is selected.
5. If the NSS pin is required in input mode, in hardware mode, connect the NSS pin to a high-level signal during the complete byte transmit sequence. In NSS software mode, set the SSM and SSI bits in the SPI_CR1 register. If the NSS pin is required in output mode, the SSOE bit only should be set. This step is not required when the TI mode is selected.
6. Set the FRF bit in SPI_CR2 to select the TI protocol for serial communications.
7. The MSTR and SPE bits must be set (they remain set only if the NSS pin is connected to a high-level signal).

Итак, разберем последовательность действий.

- CR1->BR[2:0] - биты для настройки скорости передачи
- Настройка CPOL CPHA
- Настройка длины пакета (бит DFF)
- Настройка LSB/MSB First (порядок битов, от младшего к старшему или наборот)
- настройка режима управления пином NSS (программа сама управляет или нет).
- FRF бит - Frame format - стандарт spi
- MSTR - Master - выбор роли в передаче данных
- SPE - SPI Enable
- Код можно реализовать без дополнительных структур (функционально)
- В некоторых местах допустимы отклонения в последовательности. Здесь например пункты 4 и 5 расположены в другой части.

Код будем писать сразу под датчик mpu9250 и экран oled 0.96

Тактирование:

```
void initSPI1(bool mode, uint8_t dataFrameFormat, uint16_t clkPrescaler)
{
  _RxData[num] = 0;
  _available[num] = 0;

  SPI_TypeDef *spi;
  uint8_t afNumber = 5;

  spi = SPI1;
  RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;
```

скорость + настройка nss(здесь настройка выше, чем по списку, это не играет роли):

```
if(mode == 1)
{
  spi->CR1 &= ~SPI_CR1_BR;

  switch(clkPrescaler){
    case 4: spi->CR1 |= SPI_CR1_BR_0; break;
    case 8: spi->CR1 |= SPI_CR1_BR_1; break;
    case 16: spi->CR1 |= SPI_CR1_BR_1 | SPI_CR1_BR_0; break;
    case 32: spi->CR1 |= SPI_CR1_BR_2; break;
    case 64: spi->CR1 |= SPI_CR1_BR_2 | SPI_CR1_BR_0; break;
    case 128: spi->CR1 |= SPI_CR1_BR_2 | SPI_CR1_BR_1; break;
    case 256: spi->CR1 |= SPI_CR1_BR; break;
  }
  spi->CR1 |= SPI_CR1_SSM;
  spi->CR1 |= SPI_CR1_SSI;
}
```

всё остальное:

```
if(dataFrameFormat == 16) spi->CR1 |= SPI_CR1_DFF;
else spi->CR1 &= ~SPI_CR1_DFF;

spi->CR1 &= ~SPI_CR1_CPOL;
spi->CR1 &= ~SPI_CR1_CPHA;

spi->CR1 &= ~SPI_CR1_LSBFIRST;

if(mode == 0)spi->CR2 |= SPI_CR2_RXNEIE;

if(mode == 1)spi->CR1 |= SPI_CR1_MSTR;
else spi->CR1 &= ~SPI_CR1_MSTR;

spi->CR1 |= SPI_CR1_SPE;
```

Не поверите, передача данных как и везде:

```
uint16_t writeSPI(uint16_t TxData){

  //select SPI register name
  SPI_TypeDef * spi;
  spi = SPI1;

  //write data
  while(!(spi->SR & SPI_SR_TXE));
  spi->DR = TxData;

  //read data
  while(!(spi->SR & SPI_SR_RXNE));
  return spi->DR;

}
```

Материалы курса можно найти по ссылке: https://github.com/haaroner/ezh239

Использовались материалы курса ОПД кафедры ВТ, ИТМО. Полная презентация лежит в папке docs/itmo slides part 4